

Chapter 8: Routing Algorithms

Nature Inspired Optimisation for Delivery Problems

N. Urquhart

Lecture slides – self-contained reading version

Natural Computing Series, Springer

2022

Chapter Overview

In order to solve real-world delivery and routing problems, we need a method for finding routes through road networks. This chapter provides that foundation: it explains how geospatial data can be represented as a graph, and then surveys a range of algorithms that search that graph for the shortest path between two nodes.

The chapter covers four main topics.

- ❶ **Section 8.1 — Routing Algorithms:** the core algorithms (Dijkstra and A*), their pseudocode, and step-by-step worked examples.
- ❷ **Section 8.2 — A Comparison of Basic Search Algorithms:** how the algorithms perform on real road networks (Sirmione, Italy and Edinburgh, UK), with timing data and statistical analysis.
- ❸ **Section 8.3 — Hierarchical Structures:** how splitting the road graph by road type (motorway, primary, residential) enables much faster routing on large networks.
- ❹ **Section 8.4 — Conclusions:** summary and implications for delivery-problem solvers.

Why routing algorithms matter for delivery problems

Every time a solver proposes a sequence of customer visits, the routing engine must convert that abstract sequence into an actual road distance. Without an efficient routing algorithm, the solver would be crippled by the time spent computing distances.

Graphs as Road Networks I

A **graph** is an ideal data structure for representing a road network. Each junction or location of interest becomes a *node*, and each road segment connecting two nodes becomes an *edge* (also called a *Way* in the OpenStreetMap vocabulary).

The routing algorithms described in this chapter assume that the underlying data structure exposes three methods:

Graph Interface Methods

- `linked(Node n1, Node n2)` — returns `True` if nodes `n1` and `n2` are connected by a *Way*, otherwise `False`.
- `dist(Node n1, Node n2)` — returns the distance between `n1` and `n2` if they are linked by a *Way*, otherwise returns `null`.
- `neighbours(Node n1)` — returns a list of all nodes directly linked to `n1` by a *Way* (the immediate street neighbours).

Graphs as Road Networks II

These three operations abstract away all the implementation details (adjacency matrices, linked lists, etc.) so that the routing algorithms can be written independently of the storage format.

Why abstract the graph interface?

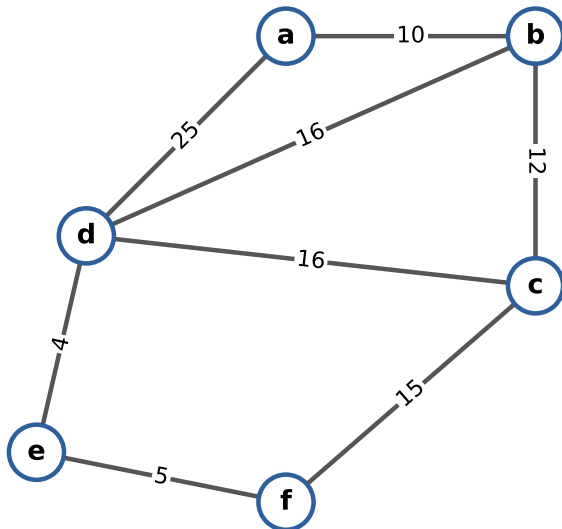
The same Dijkstra code can then run on an in-memory graph loaded from an OpenStreetMap file, or on a database-backed graph, without any changes to the algorithm itself. This mirrors standard software-engineering practice: program to an interface, not to an implementation.

Graphs as Road Networks III

The figure below shows the small weighted graph (nodes $a-f$) that will be used as a worked example throughout this chapter. Each edge is labelled with its weight (distance in arbitrary units).

Graphs as Road Networks IV

Example weighted graph (nodes a-f)



Dijkstra's Algorithm — Concept I

Dijkstra's algorithm (Dijkstra, 1959) solves the *single-source shortest-path problem*: given a starting node, find the shortest route from that node to every other node in the graph (or, in the modified variant, to a specific finish node).

The algorithm maintains two labels for every node:

- $d[n]$ — the best (shortest) distance found so far from the start node to node n . Initially set to ∞ for all nodes except the start node itself (which has $d = 0$).
- $\text{prev}[n]$ — the *predecessor* node on the best path found so far. By following the prev labels backwards from the finish node, you can reconstruct the actual route. Initially `null` for every node.

The algorithm also maintains an *unvisited* set containing all nodes that have not yet been finalised.

Core idea

At each iteration, the algorithm picks the unvisited node c with the smallest $d[c]$ (the “current” node) and examines all of c 's neighbours. For each neighbour n , it computes the distance through c : $\text{distToN} = d[c] + \text{dist}(c, n)$. If this is better than the previously recorded $d[n]$, both labels are updated. Node c is then removed from the unvisited set.

Dijkstra's Algorithm — Concept II

Why does this work?

Because we always process the node with the *smallest* accumulated distance first (a greedy choice), once a node is removed from the unvisited set its distance is guaranteed to be optimal. Any future path to that node would be longer, because all edge weights are non-negative.

Dijkstra's Algorithm — Concept III

Algorithm 16 — Dijkstra (shortest route tree variant). This version computes the shortest path from the start node to *all* other nodes, building a complete shortest-path tree.

Dijkstra's Algorithm — Concept IV

Pseudocode: Dijkstra Flood (Algorithm 16)

```
1: Procedure Dijkstra(start)
2:    $\text{dists}[] \leftarrow \infty$                                      // initialise all distances to infinity
3:    $\text{previous}[] \leftarrow \text{null}$ 
4:    $\text{current} \leftarrow \text{start}$ 
5:    $\text{dists}[\text{current}] \leftarrow 0$ 
6:    $\text{unvisited} \leftarrow \text{Graph.allNodes}()$ 
7:   while  $\text{unvisited.size} > 0$  do
8:      $\text{neighbours} \leftarrow \text{Graph.neighbours}(\text{current})$ 
9:     for Node  $n$  in neighbours do
10:       $\text{distToN} \leftarrow \text{dists}[\text{current}] + \text{Graph.dist}(\text{current}, n)$ 
11:      if  $\text{distToN} < \text{dists}[n]$  then
12:         $\text{dists}[n] \leftarrow \text{distToN}$ 
13:         $\text{previous}[n] \leftarrow \text{current}$ 
14:    $\text{unvisited.remove}(\text{current})$ 
```

Dijkstra — Modified Variant (Algorithm 17) I

The “flood” variant (Algorithm 16) is useful when you need the distance from one node to every other node — for example when pre-computing an Origin-Destination matrix. However, if you only need the distance between two specific nodes, it is wasteful to search the entire graph.

Algorithm 17 — Dijkstra Modified adds a single early-exit check: as soon as the current node equals the finish node, the algorithm returns. Only lines 14–15 differ from Algorithm 16.

Pseudocode: Dijkstra Modified (Algorithm 17)

```
1: Procedure DijkstraMod(start, finish)
2–13: (identical to Algorithm 16)
14:   if (current == finish) then return                                // early exit
15:   unvisited.remove(current)
16:   current ← findSmallest(dists, unvisited)
```

Dijkstra — Modified Variant (Algorithm 17) II

Key insight

This modification means the algorithm only searches the part of the graph that lies between the start and the finish. For a large city graph, this can reduce the number of nodes examined from tens of thousands to a few hundred.

Algorithm 18 — PrintRoute. Once Dijkstra has run, the `prev[]` array forms a backwards-linked chain from the finish node back to the start. The following procedure follows that chain to print each road name and accumulate the total distance.

Dijkstra — Modified Variant (Algorithm 17) IV

Pseudocode: PrintRoute (Algorithm 18)

```
1: Procedure PrintRoute(start, finish, prev)
2:   dist  $\leftarrow$  0
3:   current  $\leftarrow$  finish
4:   old  $\leftarrow$  previous[finish]
5:   while (current  $\neq$  null) do
6:     myWay  $\leftarrow$  Graph.getWay(current, old)
7:     print(myWay.name())
8:     dist  $\leftarrow$  dist + Graph.getDist(old, current)
9:     old  $\leftarrow$  current
10:  current  $\leftarrow$  previous[current]
11:  print("Distance = " + dist)
12: EndProcedure
```

Dijkstra — Modified Variant (Algorithm 17) V

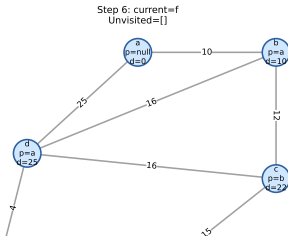
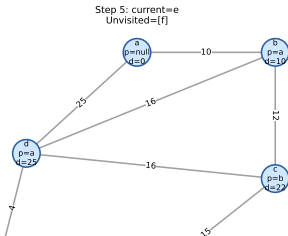
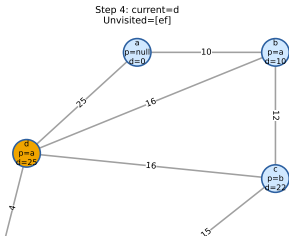
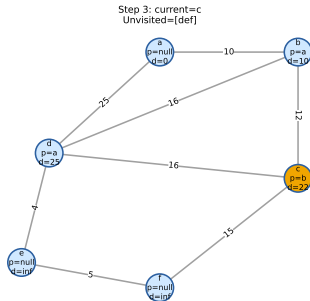
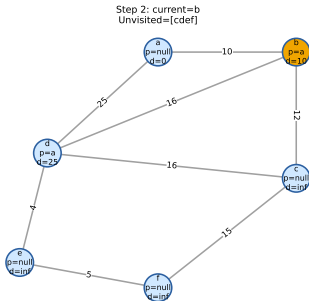
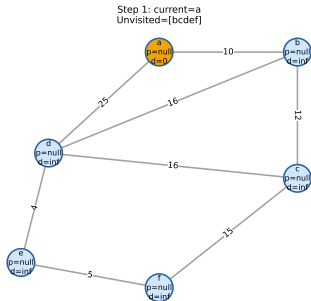
The `while` loop walks backwards from the finish node to the start by following `prev` pointers. At each step, the road name and partial distance are collected. The result is a human-readable turn-by-turn description of the route.

Dijkstra — Worked Example (Step by Step) I

We now trace Dijkstra's algorithm on the 6-node graph (a, b, c, d, e, f) with $\text{start} = a$ and $\text{finish} = f$. The figure below shows the state of the algorithm at each of the six main steps.

Dijkstra — Worked Example (Step by Step) II

Dijkstra's Algorithm -- Step-by-Step Trace (Start=a, End=f)



A* Algorithm — Concept and Motivation I

Dijkstra's algorithm has a potential weakness on large graphs: at each step it must scan the entire unvisited list to find the node with the smallest distance. More importantly, Dijkstra expands in all directions equally from the start — it does not “know” where the finish is.

A* (pronounced “A-Star”, Hart et al., 1968) addresses this by using a *heuristic* to direct the search towards the finish node. Instead of an unvisited list, A* maintains an **Open List** containing only those nodes currently under consideration.

A* Algorithm — Concept and Motivation II

The A* Heuristic: $h = d_s + d_e$

The priority score h (h) for each node in the open list is:

$$h = d_s + d_e$$

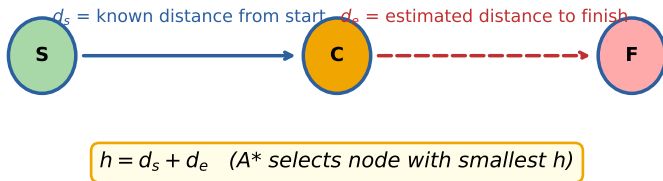
where:

- d_s — the *known* distance from the start node to this node (computed exactly, as in Dijkstra).
- d_e — the *estimated* distance from this node to the finish node, computed using the **Haversine formula** (the great-circle distance between two GPS coordinates, treating the earth as a sphere).

The algorithm always picks the node in the open list with the *smallest* h as the next current node.

A* Algorithm — Concept and Motivation III

A* Heuristic: combining known distance and estimated distance to finish



A* Algorithm — Concept and Motivation IV

How to read this formula

h balances two competing pieces of information: how far we have already travelled (d_s), and how far we still need to go (d_e). Choosing the node with the smallest h means we prefer nodes that are both *close to the start* (short d_s) *and* close to the finish (short d_e). This is why A* tends to search in a narrow “corridor” towards the destination rather than fanning out in all directions like Dijkstra.

Because the Haversine distance is an *underestimate* of the true road distance (roads are rarely straight lines), the heuristic is **admissible**: it never overestimates the true cost. An admissible heuristic guarantees that A* finds the optimal path.

A* Algorithm — Concept and Motivation V

Algorithm 19 — A-Star.

A* Algorithm — Concept and Motivation VI

Pseudocode: A* (Algorithm 19)

```
1: Procedure AStar(start, finish)
2:    $\text{dists}[] \leftarrow \infty$ 
3:    $\text{previous}[] \leftarrow \text{null}$ 
4:    $\text{current} \leftarrow \text{start}$ 
5:    $\text{dists}[\text{current}] \leftarrow 0$ 
6:    $\text{open.add}(\text{current})$  // only start is in open list
7:   while ( $\text{open.size} > 0$ ) do
8:      $\text{neighbours} \leftarrow \text{Graph.neighbours}(\text{current})$ 
9:     for Node  $n$  in neighbours do
10:       $\text{distToN} \leftarrow \text{dists}[\text{current}] + \text{Graph.dist}(\text{current}, n)$ 
11:      if ( $\text{distToN} < \text{dists}[n]$ ) then
12:        if ( $!\text{open.contains}(n)$ )
13:           $\text{open.add}(n)$  // add to open list if not there
14:           $\text{dists}[n] \leftarrow \text{distToN}$ 
```

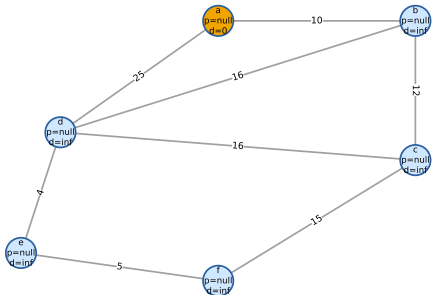
A* — Worked Example (Step by Step) I

We trace A* on the same 6-node graph, with start = a and finish = c .

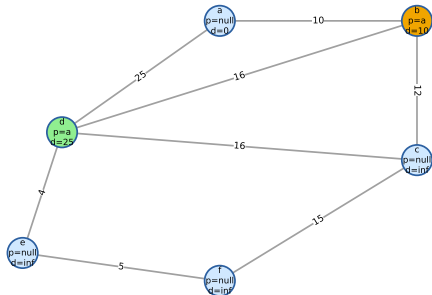
A* — Worked Example (Step by Step) II

A* Algorithm -- Step-by-Step Trace (Start=a, Finish=c, $h=ds+de$)

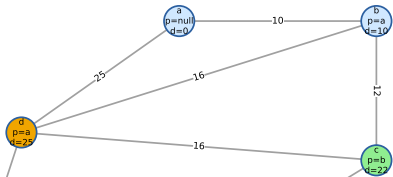
Step 1: a added to open list
open={a:0}



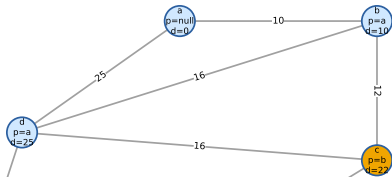
Step 2: b selected ($h=ds+de$)
openlist ordered by h



Step 3: d is current node
c and e added to open list



Step 4: finish c reached!
Trace prev labels: c<-b<-a



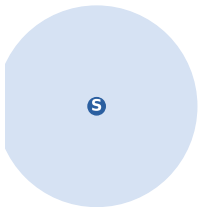
Bidirectional Search I

Another way to reduce the number of nodes examined is to run *two searches simultaneously*: one forward from the start and one in reverse from the finish. When the two search frontiers share a common node, a complete route has been found.

Bidirectional Search II

Bidirectional Search: running two algorithms simultaneously from S and F

Unidirectional Dijkstra
(searches entire reachable graph)



Bidirectional Dijkstra
(two small frontiers meet in the middle)



Bidirectional Search III

Principle of Bidirectional Search

If the search frontier expands as a circle of radius r in unidirectional search, the area searched is πr^2 . With bidirectional search, each frontier only needs radius $r/2$, so the total area searched is $2 \times \pi(r/2)^2 = \pi r^2/2$ — half as much. For a grid graph, this translates roughly to a $\sqrt{2}$ speed-up.

Bidirectional Search IV

The book implements bidirectional search as the **DijkstraBiDirectional** class. Two Dijkstra objects are created — one for the forward direction and one for the reverse. Both are run one step at a time (using the `step()` method of the abstract `RoutingAlgorithm` class), and after each pair of steps the algorithm checks whether any node is common to both frontiers.

From the book (Listing 8.4):

- A forward Dijkstra is initialised from `start`.
- A reverse Dijkstra is initialised from `finish`.
- Each iteration calls `forward.step()` and `reverse.step()`, which return the list of nodes currently being considered.
- If the intersection of these two lists is non-empty, a common node `join` has been found. Both algorithms are directed to stop at `join`, and the two half-routes are joined together.

Bidirectional Search V

Benefit and limitation

Bidirectional Dijkstra is particularly effective on large, uniform graphs. The savings are less dramatic with A*, because the heuristic already focuses the search. An `AStarBiDirectional` class is also implemented in the book following the same pattern.

Three Algorithms Under Comparison I

Section 8.2 compares three routing algorithms on real OpenStreetMap data:

The Three Algorithms

- **DijkstraFlood** — Algorithm 16: finds the shortest route from the start to *every* node in the graph. This is useful for computing full Origin-Destination (OD) matrices.
- **Dijkstra** — Algorithm 17: finds the shortest route from start to finish, halting as soon as the finish node is reached.
- **A-Star** — Algorithm 19: uses the heuristic $h = d_s + d_e$ (Haversine distance to finish) to focus the search, maintaining an open list.

Both algorithms are tested on two real road networks:

- 1 **Sirmione, Italy**: a small network of **255 nodes** connected by **60 ways** on a peninsula in Lake Garda. This is used to verify that all algorithms find the same route distances.
- 2 **Edinburgh, UK**: a large urban network of **63,267 nodes** connected by **10,612 ways**. This is the primary performance benchmark.

Three Algorithms Under Comparison II

Implementation architecture. The algorithms are structured using object-oriented programming so that they can be swapped without changing the application code.

- An abstract class `RoutingAlgorithm` defines the interface: `findPath()` (calls `step()` repeatedly), `step()` (one iteration of the algorithm), and utility methods such as `getLocations()` and `getRoadNames()`.
- Both `Dijkstra` and `AStar` are subclasses of `RoutingAlgorithm`.
- A `Route` class acts as a *facade*: the application creates a `Route` object, passes it a `RoutingAlgorithm` instance, and calls `buildRoute()`. The line `testRoute.buildRoute(new AStar())` can be changed to `new Dijkstra()` without touching any other code.

Three Algorithms Under Comparison III

Worked application code (Listing 8.5)

```
Graph myGraph = new Graph("westscot.osm");  
Route testRoute = new Route(myGraph, 291781127L, 257927392L);  
testRoute.buildRoute(new AStar());  
System.out.print("distance," + testRoute.getDist());
```

The OSM node IDs (291781127L, 257927392L) are the 64-bit integers used by OpenStreetMap to identify specific junctions in the real world.

Parsing OpenStreetMap Data I

OpenStreetMap (OSM) data is stored in an XML-based format. Each **Way** element describes a polyline (a sequence of connected **Node** elements) and can represent roads, railways, footpaths, coastlines, etc.

The implementation reads OSM data using the `BasicOSMParser` library. The key steps are:

- 1 Parse the `.osm` XML file into a Map of `Element` objects (line 12 in Listing 8.1).
- 2 Filter elements by their ID prefix: IDs beginning with “W” (line 19) are Way elements. Node elements are identified by a different prefix.
- 3 For each Way, read its `highway` attribute (line 32). Ways without a `highway` attribute are discarded (line 34–36), as are Ways belonging to highway types we are not interested in (e.g. `cycleway`, `footway`, `pedestrian` — line 37–39).
- 4 For each accepted Way, create a `RouterWay` object containing the ordered list of `RouterNode` objects along that way. Add the `RouterWay` to the Graph.

Why filter highway types?

Removing non-vehicle ways (footpaths, cycleways, etc.) reduces the graph size significantly, which in turn reduces the memory used and speeds up routing queries. For the Edinburgh dataset, the full OSM file would contain many more ways than the 10,612 used.

Parsing OpenStreetMap Data III

Creating an efficient adjacency structure: the `Matrix` class.

A naive $n \times n$ adjacency matrix for Edinburgh ($n = 63,267$) would require $63,267^2 \approx 4 \times 10^9$ entries — over 16 gigabytes of memory assuming 32 bytes per reference.

The implementation instead uses a **sparse** adjacency structure called `Matrix`. Each node has a row in a 2-D array of `NodeWay` objects, where each `NodeWay` stores a neighbour's index and the `RouterWay` connecting them. Each row is initialised to a default size of 10 (since most junctions connect to at most 10 roads) and expanded dynamically if needed.

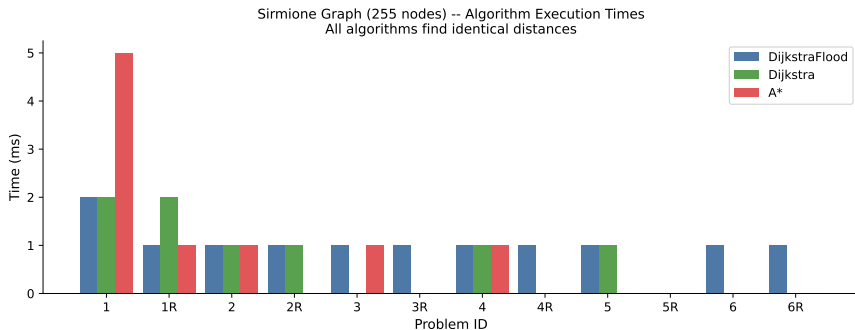
Memory saving

For Edinburgh: $63,267 \times 10 \times \text{NodeWay size} \approx 2.5 \text{ MB}$ — a reduction by a factor of over 6,400 compared to the full matrix approach.

The `Matrix.get(x, y)` method (lines 37–48 of Listing 8.2) looks up whether nodes x and y are directly linked. The `Matrix.getNeighbours(x)` method returns all `RouterWay` objects associated with node x .

Results: Sirmione (Small Graph) I

Table 8.1 from the book shows the results on the Sirmione graph (255 nodes, 60 ways). Twelve problem instances are tested; each label ending in “R” is the reverse journey of the corresponding numbered problem.



Results: Sirmione (Small Graph) II

Key observations from the Sirmione results.

- **All algorithms find identical distances.** This confirms correctness: Dijkstra and A* are provably optimal (given an admissible heuristic), and on this graph all three methods agree on every route length.
- **Execution times are very small** (0–5 ms), confirming that on a small graph the choice of algorithm barely matters in practice.
- **DijkstraFlood takes 1–2 ms** per query on this graph. Since it computes the shortest path to *every* node, running it once per node gives a complete OD matrix in total time:

$$t = n \times e = 255 \times 1.5 \text{ ms} \approx 382 \text{ ms.}$$

Here n is the number of nodes and e is the per-run execution time. For a 255-node graph, 382 ms is perfectly acceptable as a one-time pre-computation.

- **A* is occasionally slower** on the small Sirmione graph (e.g., problem 1: 5 ms vs 2 ms for Dijkstra). This overhead comes from the heuristic computation (Haversine formula) and the open-list management, which only pay off on larger graphs.

Results: Sirmione (Small Graph) III

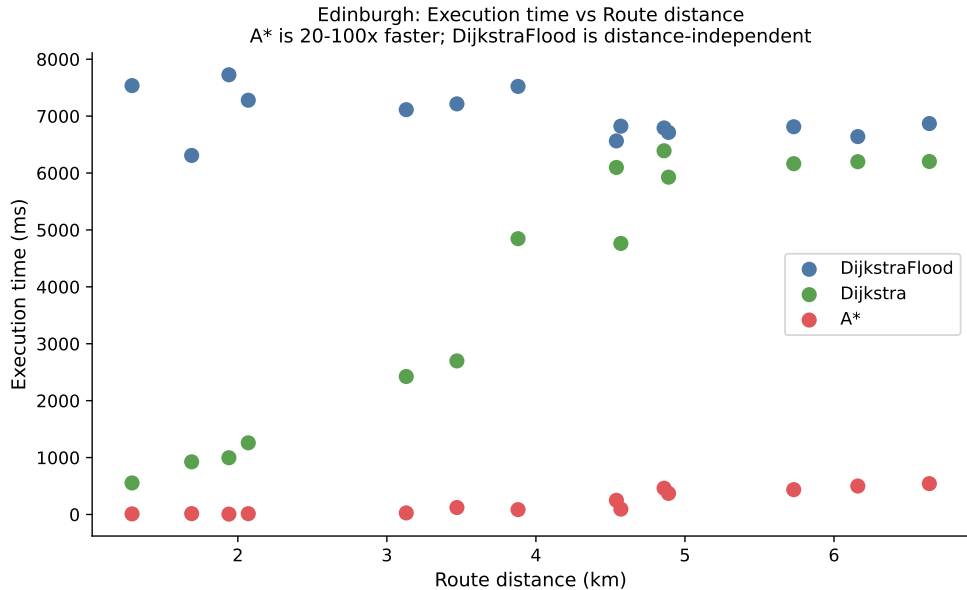
Takeaway from Sirmione

For small graphs, all algorithms give equivalent results in negligible time. The important comparison requires a large, real-world graph.

Results: Edinburgh (Large Graph) I

The Edinburgh city graph (63,267 nodes, 10,612 ways) provides a realistic large-scale benchmark. Twenty-eight distinct routes are tested (each tested in both directions, giving 56 data points). Each route is run 10 times and the average time is recorded.

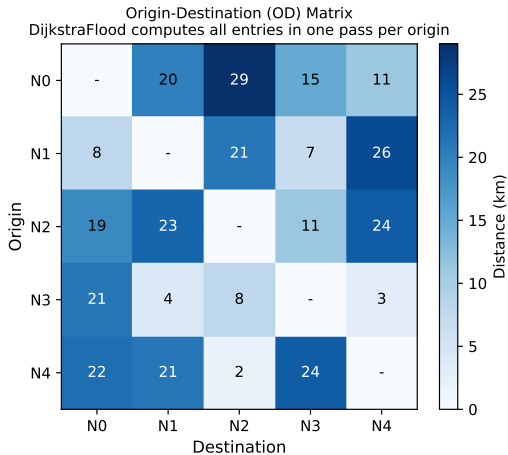
Results: Edinburgh (Large Graph) II



The Origin-Destination (OD) Matrix I

A common requirement in logistics is to pre-compute all pairwise distances between n locations. The result is an $n \times n$ *Origin-Destination (OD) matrix* where entry $[i, j]$ holds the shortest road distance from location i to location j .

The Origin-Destination (OD) Matrix II



The Origin-Destination (OD) Matrix III

Computing the OD matrix with DijkstraFlood

DijkstraFlood computes the shortest distance from one node to *all* other nodes in a single pass. Running it once for each of the n origins fills the entire matrix:

$$t_{\text{total}} = n \times e$$

where t_{total} is the total time, n is the number of nodes, and e is the execution time for one DijkstraFlood run. For Sirmione: $255 \times 1.5 \text{ ms} \approx 382 \text{ ms}$, which is negligible.

Once the OD matrix is computed, routing queries reduce to simple table lookups — far faster than running a routing algorithm at query time. This is the preferred approach when all relevant locations are known in advance (e.g., fixed depot and customer locations).

Motivation for Hierarchical Routing I

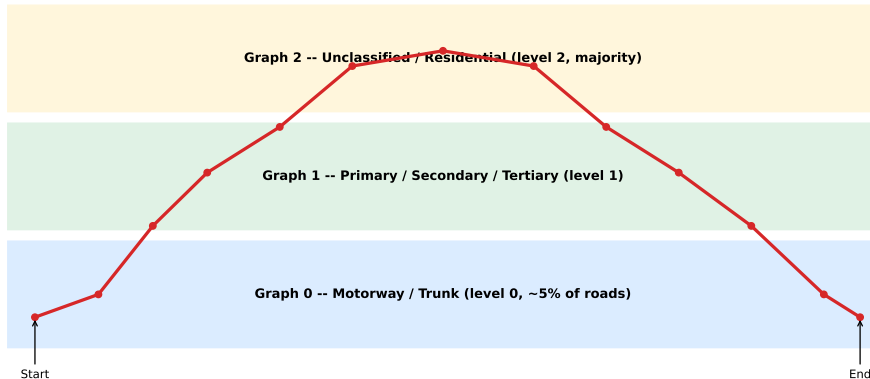
When the graph is very large (e.g., the entire road network of Scotland), even A^* may be too slow for real-time routing. The key observation is that road networks have a natural **hierarchy**:

- Long journeys predominantly use *motorways* and *trunk* roads for most of their length.
- Only the start and end segments use *local* roads (residential, unclassified).

This means we do not need to search the full graph for the bulk of the journey — a much smaller sub-graph of major roads suffices.

Motivation for Hierarchical Routing II

Hierarchical Road Network: a journey traverses local roads -> motorway -> local roads



Motivation for Hierarchical Routing III

Highway classification in OpenStreetMap.

OSM classifies each Way with a `highway` tag. The book groups these into three tiers:

Graph Level	Way types	Purpose
0	Motorway, Motorway Link, Trunk, Trunk Link	Long-distance backbone
1	Primary, Secondary, Tertiary	Medium-distance connectors
2	Unclassified, Residential	Local access (majority of roads)

The figure below shows the highway classification profile of a journey from the centre of Edinburgh to the centre of Glasgow. Note how the bulk of the journey uses motorway-class roads (Graph 0), with only short residential sections at each end.

Motivation for Hierarchical Routing IV

172

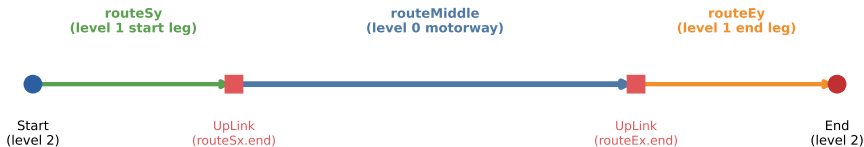
8 Routing Algorithms

```
1 public class Matrix {
2     private class NodeWay{
3         public int id;
4         public RouterWay way;
5     }
6
7     private NodeWay[][] links;
8
9     public Matrix(int size){
10         //Initialise a matrix for a graph with <size> nodes;
11         //Note that NodeWay is not sizexsize, but is initially
12         //sizex10. The node IDs used here are the position of
13         //the node within the Graph object and NOT the OSM ids.
14         links = new NodeWay[size][];
15         for (int c= 0; c < size; c++){
16             links[c] = new NodeWay[10];
17         }
18     }
19
20     public void put(int x, int y, RouterWay w){
21         //Add a link between node<x> and node<y> via Way <w>
22         NodeWay nw = new NodeWay();
23         nw.id = y;
24         nw.way = w;
25         //Find the first null entry in the matrix at row <x>
26         int oldSize = links[x].length;
27         for (int i=0; i < oldSize;i++)
```

Hierarchical Routing Algorithm I

The 3-tier routing algorithm works as follows. Given a start node (in level 2, a local road) and an end node (also in level 2), the algorithm assembles the route in three segments:

Hierarchy Algorithm: 3-segment route assembly
result = routeSy + routeMiddle + routeEy (Algorithm 21)



Hierarchical Routing Algorithm II

Algorithm 21 — Outline Hierarchy Algorithm

```
① routeSx = level[2].findLink(start)           // uplink from level 2 start
② routeSy = level[1].findLink(routeSx.end)      // uplink from level 1 start
③ routeEx = level[2].findLink(end)             // uplink from level 2 end
④ routeEy = level[1].findLink(routeEx.end)      // uplink from level 1 end
⑤ routeMiddle = level[0].path(routeSy.end, routeEy.end) // main motorway leg
⑥ result = routeSx + routeSy + routeMiddle + routeEy + routeEx
⑦ Return result
```

Hierarchical Routing Algorithm III

Explanation of each segment.

- **routeSx** is found by running a modified Dijkstra on level 2 from the start node until a *link node* (a node that also appears in level 1) is reached. This is called `DijkstraFindUpLink`.
- **routeSy** extends that search upward: from the level 2 link node, run `DijkstraFindUpLink` on level 1 until a level 0 link node (typically a motorway junction) is found.
- **routeMiddle** is the motorway/trunk leg. A standard Dijkstra (or A*) is run entirely within level 0 from the level 0 entry point to the level 0 exit point. Because level 0 contains only $\approx 5\%$ of all nodes, this search is very fast.
- **routeEy** and **routeEx** mirror the start legs, working backwards from the end node up to the motorway and then back down to the destination.
- The five segments are assembled into a `MultiRoute` object, with reversal flags applied to the end legs (which were found by searching from the end, not the start).

Hierarchical Routing Algorithm IV

Special case: start and end in same area

If the level 1 uplink nodes from start and end happen to be the same node (i.e., start and end are close to each other), no level 0 motorway search is needed. The algorithm detects this and simply runs A* directly on level 1 between the two link nodes.

Java Implementation of Hierarchical Routing I

The `HierarchyTest` class (Listing 8.8) demonstrates the 2-tier implementation. It creates the two graph levels, then runs 100 random route queries.

Setting up the levels (Listing 8.7):

Level 0 — Motorway/Trunk only

Level 0 is constructed by loading the OSM file and filtering to include *only* `motorway`, `motorway_link`, `trunk`, and `trunk_link` way types. This creates a small but fully connected backbone graph.

The `buildRoute` method (Listing 8.9):

- Lines 9–11: find the first uplink segment in level 1 by running `DijkstraFindUpLink` from the start node.
- Lines 12–15: find the last uplink segment in level 1 from the end.
- Lines 16–21: if both uplinks lead to the same level 0 node, the start and end are in the same local area. A direct A* route is computed on level 1.

Java Implementation of Hierarchical Routing II

- Lines 23–26: otherwise, the middle segment is computed on level 0 using standard Dijkstra between the two uplink end-points.
- Lines 28–33: the three segments are assembled into a `MultiRoute` object and returned.

Java Implementation of Hierarchical Routing III

Results: Edinburgh to Aberdeen (Fig. 8.11).

The 2-tier hierarchy produces a 3-part route for this ≈ 200 km journey:

- ➊ **Start leg (level 1)**: a short stretch of local roads in Edinburgh to the nearest motorway junction.
- ➋ **Middle leg (level 0)**: the majority of the journey along the A90/A956 motorway/trunk road network.
- ➌ **End leg (level 1)**: local roads in Aberdeen from the motorway exit to the destination.

Java Implementation of Hierarchical Routing IV

172

8 Routing Algorithms

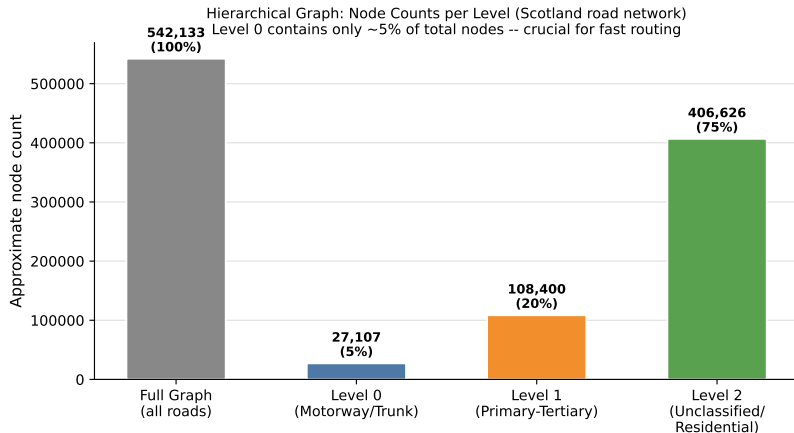
```
1 public class Matrix {
2     private class NodeWay{
3         public int id;
4         public RouterWay way;
5     }
6
7     private NodeWay[][] links;
8
9     public Matrix(int size){
10        //Initialise a matrix for a graph with <size> nodes;
11        //Note that NodeWay is not sizexsize, but is initially
12        //sizex10. The node IDs used here are the position of
13        //the node within the Graph object and NOT the OSM ids.
14        links = new NodeWay[size][];
15        for (int c= 0; c < size; c++){
16            links[c] = new NodeWay[10];
17        }
18    }
19
20    public void put(int x, int y, RouterWay w){
21        //Add a link between node<x> and node<y> via Way <w>
22        NodeWay nw = new NodeWay();
23        nw.id = y;
24        nw.way = w;
25        //Find the first null entry in the matrix at row <x>
26        int oldSize = links[x].length;
27        for (int i=0; i < oldSize;i++){
28            if (links[x][i]==null){
29                links[x][i] = nw;
30                return;
31            }
32        }
33        //if there's no space in row <x> then resize the row by 5.
34        links[x] = Arrays.copyOf(links[x], oldSize + 5);
35        links[x][oldSize+1] = nw;
```

Graph Size and Memory I

The book analyses the sizes of the graphs created for the Edinburgh-to-Aberdeen example (Table 8.5). Approximately:

- **Level 0** contains roughly 5% of the full node count. For Scotland as a whole, this still represents a non-trivial graph, but it is manageable.
- **Level 1** adds primary, secondary, and tertiary roads, bringing the total to roughly 30–40% of all nodes.
- **Level 2** (the full graph) contains 100% of nodes.

Graph Size and Memory II



Implication for the OSMFilter tool

Before loading the OSM data, the `OSMFilter` utility (Weber, 2020) is used to remove all non-road data from the OSM file. This reduces parsing time and memory consumption significantly. For a country-scale OSM file, the raw data can be many gigabytes; filtering to roads only brings this down to a manageable size.

Summary and Conclusions I

This chapter has examined the algorithms needed to find shortest routes through road networks, progressing from classical graph search to hierarchical multi-graph approaches capable of scaling to national-level networks.

Summary and Conclusions II

Summary of Routing Algorithms

Routing Algorithm Comparison

Algorithm	Finds Optimum?	Time Complexity	Space Complexity	Key Characteristic
DijkstraFlood	Yes	$O((V+E) \log V)$	$O(V)$	Full shortest-path tree
Dijkstra (mod.)	Yes	$O((V+E) \log V)$	$O(V)$	Halts at finish node
A*	Yes*	$O(E)$	$O(V)$	Heuristic reduces nodes visited
Hierarchical	Approx.	$O(\log V)$ eff.	$O(V)$ per level	Multi-graph, parallelisable
BiDirectional	Yes	$O((V+E)/2 \log V)$	$O(V)$	Two simultaneous searches

Summary and Conclusions III

Key conclusions from the chapter.

- 1 **All three basic algorithms (DijkstraFlood, Dijkstra, A*) find identical routes** on real urban road networks (Edinburgh, T-test $p = 1.0$). The choice of algorithm affects speed, not solution quality.
- 2 **A* is typically 20–100 times faster than Dijkstra** on large urban graphs, because the Haversine heuristic restricts the search to a narrow corridor towards the destination.
- 3 **DijkstraFlood is best for pre-computing OD matrices.** Its constant time per run means the full matrix can be built in $O(n)$ runs, and subsequent distance lookups are $O(1)$ table accesses.
- 4 **Hierarchical routing is necessary for national-scale networks.** By splitting the graph into motorway (5% of nodes), main-road, and local-road sub-graphs, the routing algorithm only ever searches a small fraction of the total graph for any given query. Additional benefits include parallelism and caching.
- 5 **Bidirectional search** (running forward from start and reverse from finish simultaneously) roughly halves the number of nodes searched and can be applied to both Dijkstra and A*.

Implications for delivery problem solvers

When a TSP or VRP solver iteratively evaluates millions of candidate routes, the routing engine is called for every distance query. Choosing the right routing algorithm has a multiplicative effect on the solver's overall performance:

- For small instances ($\leq$ a few hundred nodes): Dijkstra or DijkstraFlood with an OD matrix is sufficient.
- For medium instances on city graphs: A* is preferred.
- For national or continental routing: hierarchical techniques (or commercial alternatives such as GraphHopper or OpenRouteService, which implement Contraction Hierarchies) are necessary.

Summary and Conclusions V

Further reading

- Dijkstra, E.W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.
- Hart, P.E., Nilsson, N.J., Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Systems Science and Cybernetics*, 4(2), 100–107.
- Geisberger et al. (2008). Contraction hierarchies: Faster and simpler hierarchical routing. *Experimental Algorithms*, 319–333.

Python: Graph Data Structure

The following Python code mirrors the Java Graph and RouterNode classes from the book. It provides the three interface methods (linked, dist, neighbours) needed by the routing algorithms.

```
1 class RouterNode:
2     """Represents one OSM node (a junction or point of interest)."""
3     def __init__(self, node_id, lat, lon):
4         self.node_id = node_id
5         self.lat = lat      # latitude in decimal degrees
6         self.lon = lon      # longitude in decimal degrees
7         self.index = None   # position in the graph's node list
8
9 class RouterWay:
10     """Represents one OSM Way (a road segment connecting two nodes)."""
11     def __init__(self, name, nodes, length):
12         self.name = name      # road name (e.g. "Princess Street")
13         self.nodes = nodes    # list of RouterNode objects
14         self.length = length  # total length of the way in metres
15
16 class Graph:
17     """
18     Sparse graph representation of a road network.
19     Uses a dictionary of lists (adjacency list) rather than
20     a full n x n matrix to save memory.
21     """
22     def __init__(self):
23         self.nodes = {}      # node_id -> RouterNode
24         self.adjacency = {}  # node_id -> list of (neighbour_id, RouterWay)
```

Python: Haversine Distance Formula

The Haversine formula computes the great-circle distance between two points on the Earth's surface given their latitude φ (phi, degrees north) and longitude λ (lambda, degrees east). It accounts for the curvature of the Earth.

Haversine Formula

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$d = 2R \cdot \arctan 2\left(\sqrt{a}, \sqrt{1-a}\right)$$

where $\Delta\varphi = \varphi_2 - \varphi_1$ (phi, difference in latitudes in radians), $\Delta\lambda = \lambda_2 - \lambda_1$ (lambda, difference in longitudes in radians), and $R \approx 6,371$ km is the mean Earth radius.

How to read this formula

The variable a computes a “chord length” intermediate value that accounts for both the north-south separation ($\Delta\varphi$) and the east-west separation ($\Delta\lambda$, weighted by the cosines of the latitudes because longitude lines converge at the poles). The distance d then converts this chord to a surface arc length using the radius R . As a point of intuition: two locations at the same latitude differ only in $\Delta\lambda$; at the equator ($\cos\varphi \equiv 1$) this is maximally effective while at the poles ($\cos\varphi \approx 0$) longitude differences

Python: Dijkstra's Algorithm (Flood Variant)

This implementation finds the shortest distance from start to *all* reachable nodes. It uses a priority queue (min-heap) for efficiency — the standard implementation in Python via `heapq`.

```
1  import heapq
2
3  def dijkstra_flood(graph, start_id):
4      """
5      Dijkstra flood fill: returns (dists, prev) dicts for all reachable nodes.
6      graph    : a Graph object with .neighbours(node_id) method
7      start_id: the OSM node ID of the starting node
8      """
9      # Initialise all distances to infinity
10     dists = {node_id: float('inf') for node_id in graph.nodes}
11     prev  = {node_id: None          for node_id in graph.nodes}
12     dists[start_id] = 0
13
14     # Priority queue entries are (distance, node_id)
15     pq = [(0, start_id)]
16
17     while pq:
18         current_dist, current = heapq.heappop(pq)
19
20         # Skip if we already found a shorter path (stale entry in heap)
21         if current_dist > dists[current]:
22             continue
23
24         for neighbour, edge_dist in graph.neighbours(current):
```

Python: Dijkstra Modified (Early Exit)

The modified Dijkstra halts as soon as the finish node is popped from the priority queue. This is equivalent to the modified Algorithm 17 from the book.

```
1 def dijkstra_mod(graph, start_id, finish_id):
2     """
3     Modified Dijkstra: stops as soon as finish_id is settled.
4     Returns (distance, route_node_ids).
5     """
6     dists = {node_id: float('inf') for node_id in graph.nodes}
7     prev = {node_id: None for node_id in graph.nodes}
8     dists[start_id] = 0
9
10    pq = [(0, start_id)]
11
12    while pq:
13        current_dist, current = heapq.heappop(pq)
14
15        # Early exit: finish node has been settled
16        if current == finish_id:
17            break
18
19        if current_dist > dists[current]:
20            continue # stale heap entry -- skip
21
22        for neighbour, edge_dist in graph.neighbours(current):
23            dist_to_n = dists[current] + edge_dist
24            if dist_to_n < dists[neighbour]:
```

Python: A* Algorithm

The Python A* implementation closely mirrors Algorithm 19 from the book. The priority queue orders nodes by $h = d_s + d_e$, where d_e is the Haversine distance from the node to the finish.

```
1  def astar(graph, start_id, finish_id):
2      """
3      A* algorithm using Haversine great-circle distance as heuristic.
4      Returns (distance, route_node_ids).
5      """
6      finish_node = graph.nodes[finish_id]
7
8      def heuristic(node_id):
9          """d_e: Haversine distance from node to finish (underestimate)."""
10         n = graph.nodes[node_id]
11         return haversine(n.lat, n.lon, finish_node.lat, finish_node.lon)
12
13     dists = {node_id: float('inf') for node_id in graph.nodes}
14     prev = {node_id: None for node_id in graph.nodes}
15     dists[start_id] = 0
16
17     # Priority queue entries: (h_value, node_id)
18     # h = d_s + d_e = dists[node_id] + heuristic(node_id)
19     open_set = {start_id}
20     pq = [(heuristic(start_id), start_id)]
21
22     while pq:
23         h_current, current = heapq.heappop(pq)
```

Python: Comparing Dijkstra vs. A*

The following demonstration script runs both algorithms on a small synthetic graph and compares their results and node-visit counts.

```
1  import time
2
3  def demo_comparison():
4      """
5      Build a small test graph and compare Dijkstra vs A*.
6      This replicates the Sirmione-style experiment from the book.
7      """
8      # Build a simple 6-node graph matching the book example
9      # (nodes assigned approximate GPS coords for heuristic use)
10     g = Graph()
11     coords = {
12         'a': (45.61, 10.60), 'b': (45.61, 10.62),
13         'c': (45.60, 10.62), 'd': (45.60, 10.59),
14         'e': (45.59, 10.59), 'f': (45.59, 10.61),
15     }
16     for nid, (lat, lon) in coords.items():
17         node = RouterNode(nid, lat, lon)
18         node.index = nid
19         g.add_node(node)
20
21     # Add edges with synthetic distances (matching book weights x 100m)
22     edges = [
23         ('a', 'b', 1000), ('a', 'd', 2500), ('b', 'c', 1200), ('b', 'd', 1600),
24         ('c', 'd', 1600), ('c', 'f', 1500), ('d', 'e', 400), ('e', 'f', 500),
```

Python: Hierarchical Routing (Simplified)

A simplified 2-level hierarchical router illustrating the concept from Section 8.3. In practice, levels are separate Graph objects loaded with different OSM highway types.

```
1 def find_uplink(graph, start_id, upper_graph):
2     """
3     Run Dijkstra from start_id until a node that exists in upper_graph
4     is found. Returns (route_to_link_node, link_node_id).
5     This is the 'DijkstraFindUpLink' concept from the book.
6     """
7     dists = {nid: float('inf') for nid in graph.nodes}
8     prev = {nid: None for nid in graph.nodes}
9     dists[start_id] = 0
10    pq = [(0, start_id)]
11
12    while pq:
13        d, current = heapq.heappop(pq)
14        # Check: is this node a link node (present in the upper graph)?
15        if current != start_id and current in upper_graph.nodes:
16            route = get_route(prev, start_id, current)
17            return dists[current], route, current
18        if d > dists[current]:
19            continue
20        for neighbour, edge_dist in graph.neighbours(current):
21            dist_to_n = dists[current] + edge_dist
22            if dist_to_n < dists[neighbour]:
23                dists[neighbour] = dist_to_n
24                prev[neighbour] = current
```

Chapter Summary I

This chapter has provided the algorithmic foundations needed to find routes through real-world road networks, progressing from classical graph search algorithms to hierarchical multi-graph techniques.

What you should now be able to do:

- 1 **Describe Dijkstra's algorithm** — both the flood variant that builds a full shortest-path tree and the modified variant that halts at a specific finish node. You should be able to trace the algorithm step by step on a small graph.
- 2 **Describe A*** — how it differs from Dijkstra (the Open List and the heuristic $h = d_s + d_e$), why the Haversine distance is admissible, and why A* is faster on large graphs.
- 3 **Explain the OD matrix** and why DijkstraFlood is the natural algorithm for pre-computing it.
- 4 **Explain hierarchical routing** — why road networks have a natural hierarchy, how splitting the graph by highway type enables much faster routing, and the three-segment route assembly (local start leg + motorway middle + local end leg).
- 5 **Implement routing in Python** using a priority queue (min-heap), and understand the correspondence between the Python and Java implementations described in the book.

Chapter Summary II

Computational complexity at a glance:

Algorithm	Optimal?	Time	Space	When to use
DijkstraFlood	Yes	$O((V + E) \log V)$	$O(V)$	OD matrices
Dijkstra (mod)	Yes	$O((V + E) \log V)$	$O(V)$	Single pair
A*	Yes*	$O(E)$ best case	$O(V)$	Large graphs
Bidirectional	Yes	$\approx O(\frac{V+E}{2} \log V)$	$O(V)$	Symmetric graphs
Hierarchical	Approx	$O(\log V)$ eff.	$O(V)$ per level	National scale

* A* is optimal when the heuristic is admissible (never overestimates).

Final message

Selecting the right routing algorithm is not just an academic exercise — it directly determines whether a delivery optimisation solver can evaluate routes fast enough to be useful in practice. A* on a city graph and hierarchical routing on a national graph are the two most important tools in this space.